

数据仓库开发工具Hive

Grissom | 2025年11月

目录 >

CONTENTS

- 1 Hive 简介
- 2 HQL 开发
- 3 Hive JDBC
- 4 Hive 函数
- 5 Hive 性能优化



1

chapter

Hive 简介

➤ 诞生背景

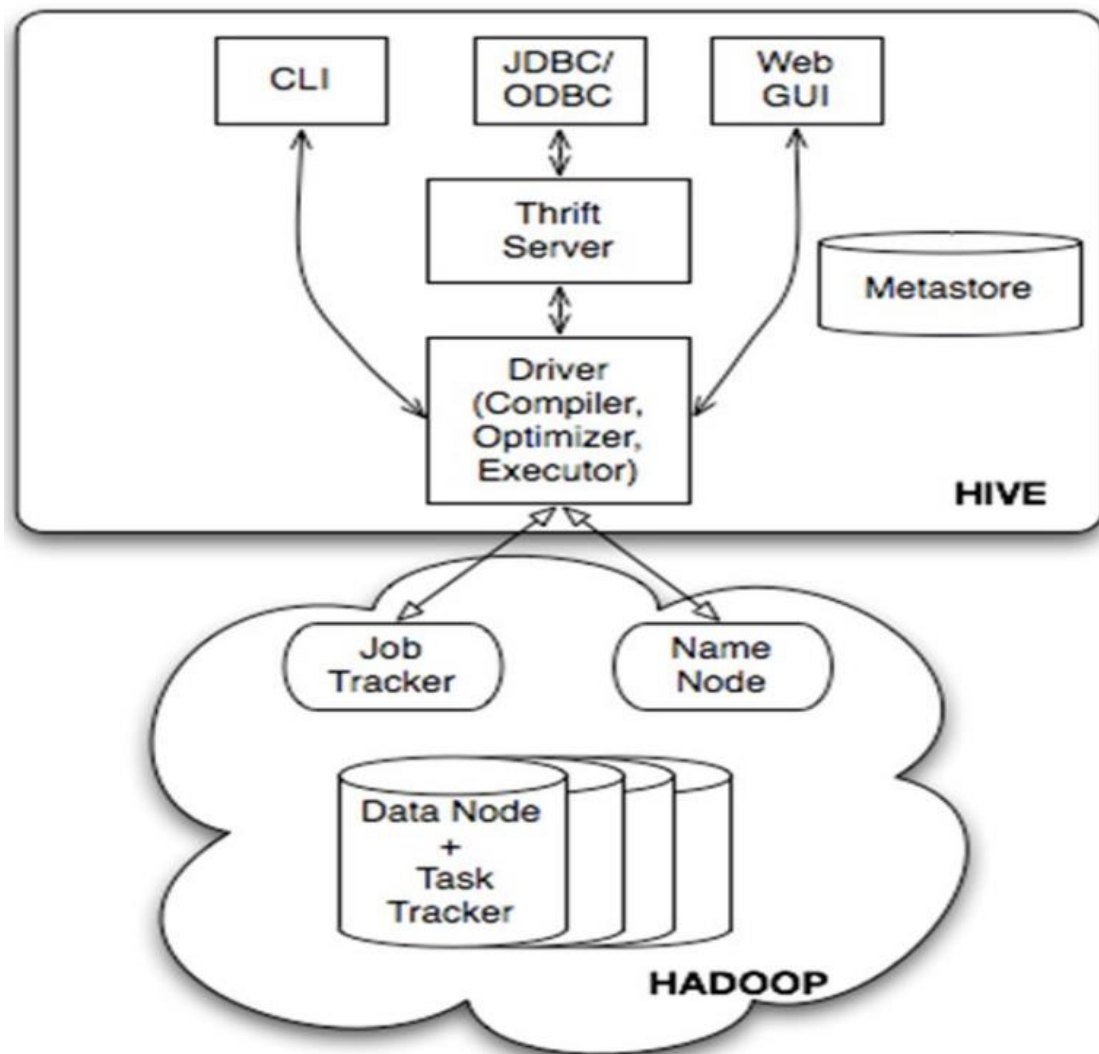
- MapReduce开发成本太高
- 类SQL语句的大数据编程

➤ Hive

- 早期由Facebook开发，后来由Apache软件基金会开发，并成为其顶级项目
- 基于Hadoop的一个数据仓库工具
- 核心思想：将结构化的数据文件映射为一张数据库表，并提供简单的sql查询功能，可以将sql语句转换为MapReduce任务进行运行



➤ Hive 架构



➤ Hive 架构

1. 用户接口主要有三个：CLI，Client 和 WUI。其中最常用的是CLI，Cli启动的时候，会同时启动一个Hive副本。Client是Hive的客户端，用户连接至Hive Server。在启动 Client模式的时候，需要指出Hive Server所在节点，并且在该节点启动Hive Server。 WUI是通过浏览器访问Hive。
2. Hive将元数据存储在数据库中，如mysql、derby。Hive中的元数据包括表的名字，表的列和分区及其属性，表的属性（是否为外部表等），表的数据所在目录等。
3. 解释器、编译器、优化器完成HQL查询语句从词法分析、语法分析、编译、优化以及查询计划的生成。生成的查询计划存储在HDFS中，并在随后有MapReduce调用执行。
4. Hive的数据存储在HDFS中，大部分的查询、计算由MapReduce完成（包含*的查询，比如select * from tbl不会生成MapRedcue任务）。



2 chapter

HQL 开发

- ✓ 数据类型支持
- ✓ 数据操作语言DML
- ✓ 数据定义语言DDL

➤ 数据类型支持

数据类型	描述	示例
TINYINT	1字节（8位）有符号整数，从-128到127	1
SMALLINT	2字节（16位）有符号整数，从-32768到32767	1
INT	4字节（32位）有符号整数，从-2147483648到2147483647	1
BIGINT	8字节（64位）有符号整数，从-9223372036854775808到9223372036854775807	1
FLOAT	4字节单精度浮点数	1.0
DOUBLE	8字节双精度浮点数	1.0
DECIMAL	不可变的、任意精度的、有符号的十进制数	1.012,1e+44
BOOLEAN	TRUE/FALSE	TRUE
STRING	字符串	'a', "a"
VARCHAR	可变长度的字符串	'a', "a"
DATE	日期，格式：'YYYY-MM-DD'，从'0001-01-01'到'9999-12-31'	'2014-01-01'
TIMESTAMP	时间戳，表示日期和时间，格式：'YYYY-MM-DD HH:MM:SS.ffffffff'，可达到小数点后9位精度（纳秒级别）	'2014-01-01 00:00:00'

➤ 数据类型支持

数据类型	描述	示例
ARRAY	一组有序字段，所有字段的数据类型必须相同	ARRAY(1,2)
MAP	一组无序的键/值对，键的类型必须是原生数据类型，值的类型可以是原生或复杂数据类型；同一个MAP的键的类型必须相同，值的类型也必须相同	MAP('a',1;'b',2)
STRUCT	一组命名的字段，字段的数据类型可以不同	STRUCT('a',1,1.0)
UNIONTYPE	可以将上面的数据类型组合在一起	UNIONTYPE<int, double, array<string>>

➤ 数据导入

1. LOAD DATA

```
LOAD DATA [LOCAL] INPATH 'filepath'  
[OVERWRITE] INTO TABLE tablename [PARTITION (partcol1=val1, partcol2=val2 ...)]
```

- INPATH选项

- <path>是文件存储路径，如果标注LOCAL，指向本地磁盘路径，不标则指向HDFS路径
- <path>既可指向文件也可指向目录，系统会将指定文件或目录下所有的文件移入表内
- <path>是HDFS路径时，执行操作的用户必须是<path>的Owner，同时Inceptor用户必须有读写权限

- 将文件中的数据导入已存在的表或分区

- 仅将数据文件移动到表或分区的目录中，不会对数据进行任何处理，如分桶、排序

➤ 数据导入

2. INSERT...SELECT...FROM

```
INSERT OVERWRITE | INTO TABLE tablename1 [PARTITION (partcol1=val1, partcol2=val2 ...)  
[IF NOT EXISTS]] select_statement1 FROM from_statement;
```

- 将查询结果导入到相应表当中
- 常用于分析的SQL语句中
- 除支持以上基本用法，还支持多个insert方式：

```
FROM from_statement  
INSERT OVERWRITE TABLE tablename1 [PARTITION (partcol1=val1, ...) [IF NOT EXISTS]] select_statement1  
[INSERT OVERWRITE TABLE tablename2 [PARTITION ... [IF NOT EXISTS]] select_statement2]  
[INSERT INTO TABLE tablename2 [PARTITION ...] select_statement2] ...;
```

➤ 数据导入

3. INSERT INTO VALUES

```
INSERT INTO TABLE tablename [PARTITION (partcol1[=val1], partcol2[=val2] ...)]  
VALUES values_row [, values_row ...]
```

- 不支持复杂数据类型 (array, map, struct, union)
- 不推荐使用该方法, 进行数据导入

➤ 数据导出

```
INSERT OVERWRITE [LOCAL] DIRECTORY <directory>  
  [ROW FORMAT row_format] [STORED AS file_format]  
  SELECT ... FROM ...
```

row_format

```
: DELIMITED [FIELDS TERMINATED BY char [ESCAPED BY char]] [COLLECTION ITEMS TERMINATED BY char]  
  [MAP KEYS TERMINATED BY char] [LINES TERMINATED BY char]  
  [NULL DEFINED AS char]
```

- 支持数据导出到本地以及HDFS，标注LOCAL代表导出到本地磁盘，不标代表导出到HDFS
- 将查询结果导出到本地目录，可能生成多个文件
- 与导入数据不一样，不能用Insert Into导出数据，只能用Insert Overwrite
- DIRECTORY指定数据导出的文件目录
- ROW FORMAT指定文件的行格式，STORED AS指定文件格式，不指定使用默认值

➤ 创建数据库

```
CREATE DATABASE [IF NOT EXISTS] <database_name>  
  [COMMENT '<database_comment>']  
  [WITH DBPROPERTIES ('<property_name>'='<property_value>', ...)];
```

➤ 删除数据库（先删表后删库）

```
DROP DATABASE [IF EXISTS] <database_name>;
```

➤ 修改数据库属性

```
ALTER DATABASE <database_name> SET DBPROPERTIES ('<property_name>'='<property_value>', ...);
```

➤ 列出所有数据库

```
SHOW DATABASES;
```

➤ 切换数据库、查看数据库详情

```
USE <database_name>;  
DESCRIBE <database_name>;
```

➤ 完整建表语法规则

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]table_name
  [(col_name data_type [COMMENT col_comment], ... [constraint_specification])]
  [COMMENT table_comment]
  [PARTITIONED BY (col_name data_type [COMMENT col_comment], ...)]
  [CLUSTERED BY (col_name, col_name, ...) [SORTED BY (col_name [ASC|DESC], ...)]
    INTO num_buckets BUCKETS]
  [SKEWED BY (col_name, col_name, ...) ]
    ON ((col_value, col_value, ...), (col_value, col_value, ...), ...)
  [STORED AS DIRECTORIES]
  [
    [ROW FORMAT row_format]
    [STORED AS file_format]
    | STORED BY 'storage.handler.class.name' [WITH SERDEPROPERTIES (...)]
  ]
  [LOCATION hdfs_path]
  [TBLPROPERTIES (property_name=property_value, ...)];
```

➤ 完整建表语法规则

```
data_type
: primitive_type
| array_type
| map_type
| struct_type
| union_type

array_type
: ARRAY < data_type >

map_type
: MAP < primitive_type, data_type >

struct_type
: STRUCT < col_name : data_type [COMMENT col_comment], ...>

union_type
: UNIONTYPE < data_type, data_type, ... >
```

➤ 完整建表语法规则

```
row_format
: DELIMITED
  [FIELDS TERMINATED BY char [ESCAPED BY char]]
  [COLLECTION ITEMS TERMINATED BY char]
  [MAP KEYS TERMINATED BY char]
  [LINES TERMINATED BY char]
  [NULL DEFINED AS char]
| SERDE serde_name
  [WITH SERDEPROPERTIES (property_name=property_value, property_name=property_value, ...)]

file_format:
: SEQUENCEFILE
| TEXTFILE -- (Default, depending on hive.default.fileformat configuration)
| RCFILE | ORC | PARQUET | AVRO | JSONFILE
| INPUTFORMAT input_format_classname OUTPUTFORMAT output_format_classname
```

➤ 示例：

- 要求：建议对应的Hive表，存放HDFS中采集到人员信息数据。
- 数据样本：

```
1,tom,18,music-game-driver,std_addr  
2,jolin,21,music-movie,std_addr:beijing-work_addr:shanghai-addr:tokyo  
3,tony,33,book-game-food,std_addr:beijing-work_addr:xian  
4,lilei,12,scl_addr:xizhimen-home_addr:null  
5,hanmeimei,12,scl_addr:xizhimen  
6,baby,3,food,addr:tianjing
```

➤ 示例：

- SQL Demo:

```
CREATE TABLE psn(  
  id INT,  
  name STRING,  
  age INT,  
  likes ARRAY<STRING>,  
  address MAP<STRING,STRING>  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
COLLECTION ITEMS TERMINATED BY '-'  
MAP KEYS TERMINATED BY ':'  
LINES TERMINATED BY '\n';  
  
LOAD DATA INPATH '/tmp/dataset/psn.1' INTO TABLE psn;
```

➤ 删除表

- 对于内表，表的元数据和数据都会被删除；对于外表，只删除元数据

```
DROP TABLE <table_name>;
```

➤ 修改表

```
/* 表重命名 */
```

```
ALTER TABLE <table_name> RENAME TO <new_table_name>;
```

```
/* 修改表属性 */
```

```
ALTER TABLE <table_name> SET TBLPROPERTIES ('<property_name>' = '<property_value>' ... );
```

```
ALTER TABLE <table_name> SET SERDEPROPERTIES ('<property_name>' = '<property_value>' ... );
```

```
ALTER TABLE <table_name> SET LOCATION '<new_location>';
```

```
/* 增加、删除、修改、替换列 */
```

```
ALTER TABLE <table_name> ADD COLUMNS (<col_spec> [, <col_spec> ...])
```

```
ALTER TABLE <table_name> DROP [COLUMN] <col_name>
```

```
ALTER TABLE <table_name> CHANGE <col_name> <new_col_name> <new_col_type>
```

```
ALTER TABLE <table_name> REPLACE COLUMNS (<col_spec> [, <col_spec> ...])
```

➤ 清空表

- 清空表中的数据，但不删除元数据，该操作作用于内表，不能用于外表

```
TRUNCATE TABLE <table_name>;
```

➤ 查看表详情

```
DESCRIBE TABLE <table_name>;
```

➤ 列出当前数据库的所有表

```
SHOW TABLES ;
```

➤ 内部表

- 通过自定义Schema来创建内表
- 存储在数据库目录下的同名子目录中
- 删除内表时，同时删除表中数据和元数据

```
CREATE TABLE <table_name>  
[(<col_name> <data_type> [, <col_name> <data_type> ...]);
```

➤ 外部表

- 创建外表时，一定要指定数据存储位置
- 常用于数据导入，先建外表，再将外表Location设置为数据文件所在目录
- 删除外表时，只删除元数据，表中数据保留

```
CREATE EXTERNAL TABLE <table_name>  
(<col_name> <data_type> [, <col_name> <data_type> ...])  
LOCATION '<file_path>';
```

➤ Create Table Like

- 通过已存在的表或视图来创建表，只复制Schema，不复制数据

```
CREATE TABLE <table_name>  
  LIKE <existing_table_or_view_name>;
```

- 例如：

```
CREATE TABLE new_tbl  
  LIKE exit_tbl;
```

➤ Create Table As Select (CTAS)

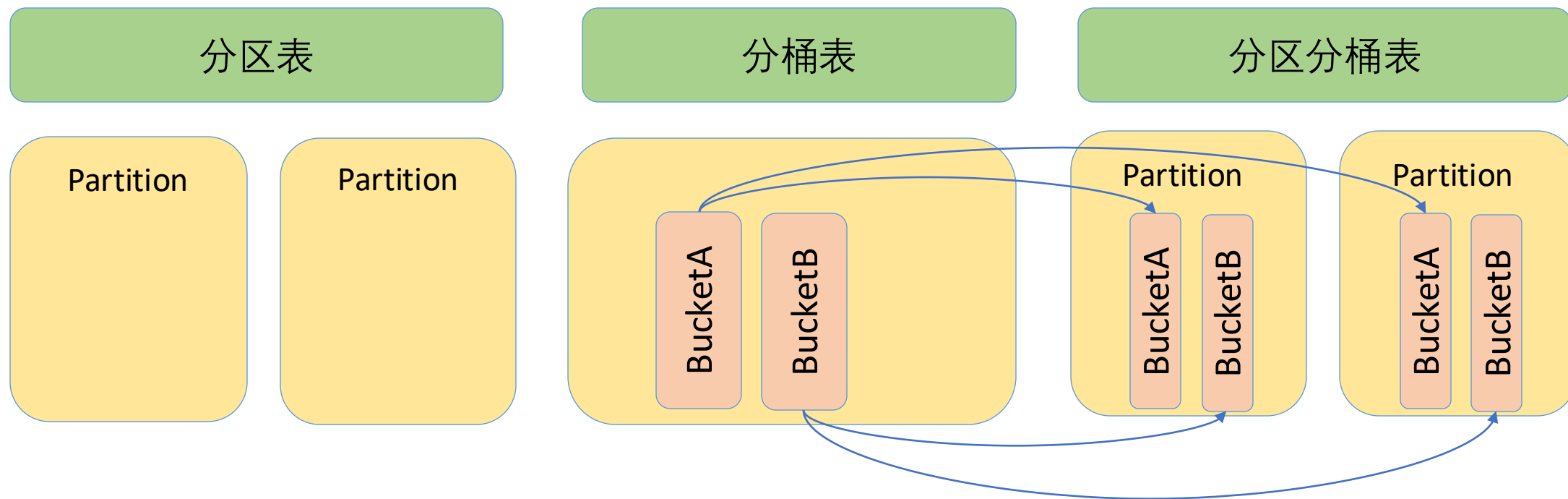
- 通过查询结果来创建表，既复制Schema，又复制数据

```
CREATE TABLE <table_name>  
  AS SELECT <select_statement>;
```

- 示例：

```
CREATE TABLE new_key_value_store  
  ROW FORMAT SERDE "org.apache.hadoop.hive.serde2.columnar.ColumnarSerDe"  
  STORED AS RCFile  
  AS  
  SELECT (key % 1024) new_key, concat(key, value) key_value_pair  
  FROM key_value_store  
  SORT BY new_key, key_value_pair;
```

➤ 分区 & 分桶



➤ 分区 (Partition)

- 含义：通过特定条件（相等或大于小于）将表的数据分发到分区目录中，或者将分区中的数据分发到子分区目录中。

➤ 分区的作用

- 减少不必要的全表扫描，提升查询效率，如
Select ... Where



➤ 分区表

- 建表语法规则：

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]table_name
  [(col_name data_type [COMMENT col_comment], ... [constraint_specification])]
  [COMMENT table_comment]
PARTITIONED BY (col_name1 data_type, col_name2 data_type [COMMENT col_comment], ...)
  [ROW FORMAT row_format]
  [STORED AS file_format]
  [LOCATION hdfs_path]
  [TBLPROPERTIES (property_name=property_value, ...)];
```

- 查询指定分区数据：

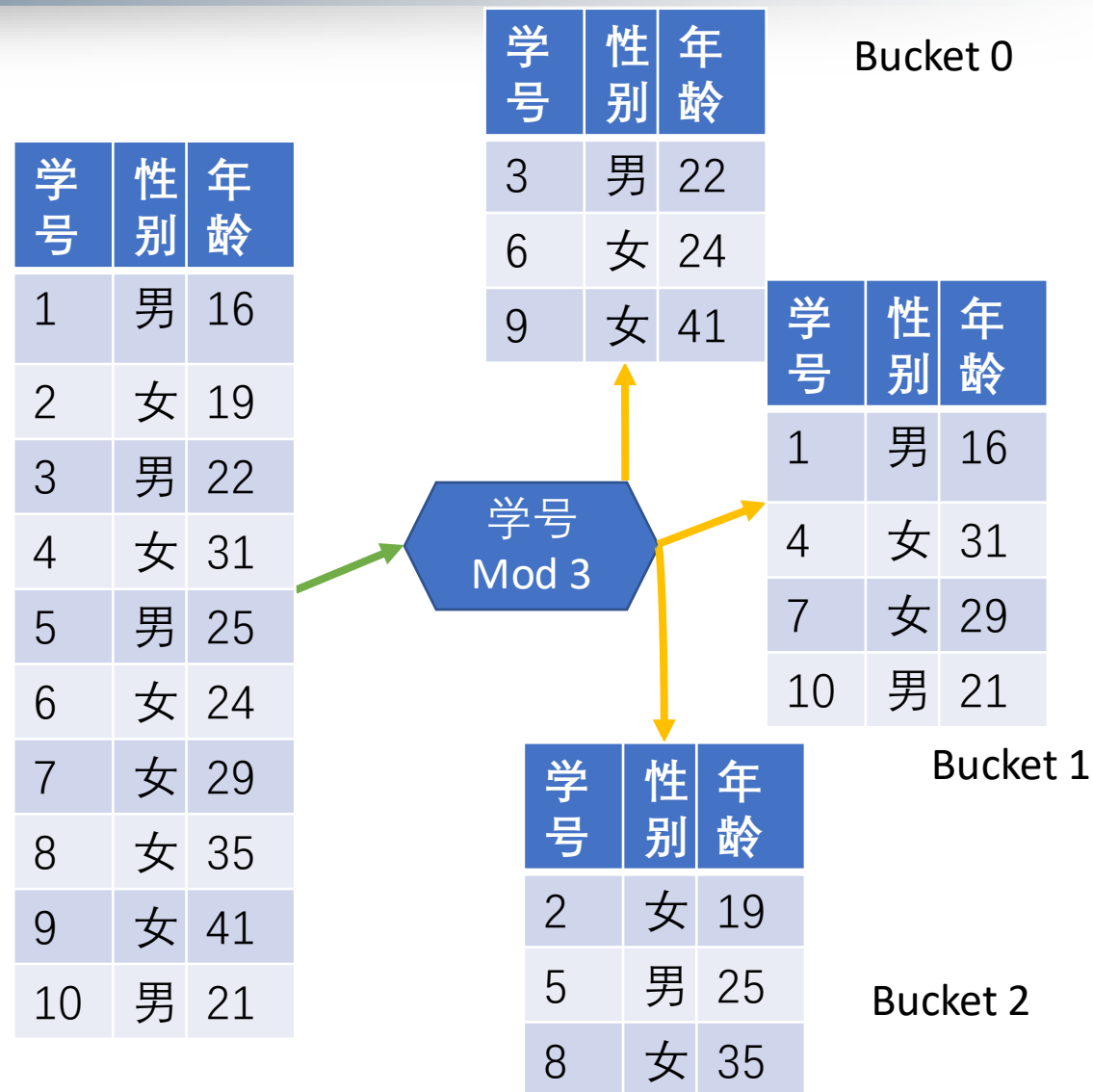
```
SELECT select_expr, select_expr, ... FROM table_reference
  WHERE partition_key = p_value [ AND ...]
```

➤ 分桶 (Bucket)

- 含义：通过分桶键哈希取模（ $\text{key hashcode \% N}$ ）的方式，将表或分区中的数据随机、均匀地分发到N个桶中，桶数N一般为质数，桶编号为0, 1, ..., N-1

➤ 分桶的作用:

- ① 提高取样效率：从桶中直接抽取数据
- ② 提高Join查询效率：不用全表遍历，对应的桶Join就可以了
- ③ 分桶操作使得如GROUP BY以及特定场景下的JOIN能够在一个Stage中完成，避免了Shuffle过程。减少Shuffle数据量



➤ 分桶表

- INTO BUCKETS设定桶的数量，SORTED BY设定桶的排序，默认升序

```
CREATE [EXTERNAL] TABLE <table_name>
  (<col_name> <data_type> [, <col_name> <data_type> ...])
  [PARTITIONED BY ...]
  CLUSTERED BY (<col_name>)
    [SORTED BY (<col_name> [ASC | DESC] [, <col_name> [ASC | DESC]...])]
  INTO <num_buckets> BUCKETS
  [ROW FORMAT <row_format>]
  [LOCATION '<file_path>']
  [TBLPROPERTIES ('<property_name>'='<property_value>', ...)];
```

➤ 开启分桶表支持

- set hive.enforce.bucketing=true;

- 默认：false；设置为true之后，mr运行时会根据bucket的个数自动分配reduce task个数。（用户也可以通过mapred.reduce.tasks自己设置reduce任务个数，但分桶时不推荐使用）
- 注意：一次作业产生的桶（文件数量）和reduce task个数一致。

3
chapter

Hive JDBC

- **JDBC** (Java DataBase Connectivity)
- 是一种用于执行SQL语句的Java API, 可以为多种数据库提供统一访问, 由一组用Java语言编写的类和接口组成, 定义了一种访问标准, 使开发人员能够基于这种标准编写数据库应用程序。
- 开发流程:
 1. 获取驱动
 2. URL连接串
 3. 建立连接
 4. 获取Statement
 5. 执行sql
 6. 返回结果
 7. 关闭连接

➤ Demo

```
public static void main(String[] args) throws SQLException {  
  
    // 驱动  
    String driverName = "org.apache.hive.jdbc.HiveDriver";  
    // URL连接  
    String jdbcURL = "jdbc:hive2://192.168.0.205:10000/default";  
  
    try {  
        // 加载JDBC驱动  
        Class.forName(driverName);  
    } catch (ClassNotFoundException e) {  
        e.printStackTrace();  
    }  
  
    // 建立连接，获取Connection  
    Connection conn = DriverManager.getConnection(jdbcURL);  
    // 获取Statement  
    Statement stmt = conn.createStatement();  
}
```

➤ Demo

```
// 创建表
String createSQL = "CREATE TABLE torctbl (id STRING, name STRING, age INT) ";
stmt.execute(createSQL);

Random r = new Random();

String insertSQL = "INSERT INTO torctbl(id, name, age) VALUES (?, ?, ?)";
// 获取PreparedStatement
PreparedStatement pstmt = conn.prepareStatement(insertSQL);
for (int i = 0; i < 100; i++) {
    pstmt.setString(1, String.format("%04d", i));
    pstmt.setString(2, "xiaoming"+i);
    pstmt.setInt(3, r.nextInt(20));
    pstmt.addBatch();
}
pstmt.executeBatch();
```

➤ Demo

```
String querySQL = "select id, name, age from torctbl where age = 6 limit 10";

// 提交查询SQL请求，返回查询结果
ResultSet rs = stmt.executeQuery(querySQL);

while (rs.next()) {
    System.out.println("ID:" + rs.getString(1)
        + " - Name:" + rs.getString(2) + " - Age:" + rs.getInt("age"));
}

rs.close();
stmt.close();
conn.close();
}
```


4
chapter

Hive 函数

➤ Hive允许用户编写自己定义的函数，在查询中能结合SQL语句一起使用，更好地表达复杂的业务逻辑。

➤ 函数分类：

1. UDF (User Defined Function)

即用户自定义函数，以单个数据行为参数，输出单个数据行。

一进一出，例如：trim, to_char, next_day

2. UDAF (User Defined Aggregation Function)

即自定义聚合函数，以多个数据行为参数，输出一个数据行。

多进一出，例如：count, sum, avg

3. UDTF (User Defined Table Function)

即用户自定义表函数。用来实现多（行/列）对多（行/列）数据的处理逻辑。以一个数据行为参数，输出多个数据行为一个表作为输出。

一进多出，例如：explode

➤ 自定义函数开发流程：

① 代码开发

继承相应UDF、UDAF、UDTF类实现各自方法，在方法中定义好用户需要实现的函数功能。

② 将工程打成jar包

注意：

- jar包不应该包含，hive开头的任何jar包
- jar包中的自定义UDF类名，不能和现有UDF类，在“包名+类名”上完全相同
- 如果在工程打包时添加了hive等jar包，需要保证添加的hive和hadoop等依赖包，必须保证和集群中jar包完全一致，防止版本冲突问题

③ 上传jar包

支持上传到服务器本地以及HDFS

- 上传到本地，需要将jar包上传到HiveServer节点上
- 上传到HDFS，推荐使用这种方式。

➤ 自定义函数开发流程:

④ 添加jar包

注意：在Hive重启之前，不能add 两个名字一样的jar；也不能add 两个名字不一样，但是包含了相同Class 的jar。

命令：add jar hdfs://{NameService}/tmp/udf/udfProject.jar;

⑤ 创建函数

- 临时函数 -- Hive Session有效，Session结束后该函数失效
- 永久函数

➤ 添加、查看、删除Jar包

```
ADD JAR[S] <filepath1> [<filepath2>]*  
LIST JAR[S] [<filepath1> <filepath2> ..]  
DELETE JAR[S] [<filepath1> <filepath2> ..]
```

➤ 创建、删除临时函数

```
CREATE TEMPORARY FUNCTION <function_name> AS "<class_name>";  
DROP TEMPORARY FUNCTION [IF EXISTS] <function_name>;
```

➤ 创建、删除永久函数

```
CREATE FUNCTION <function_name> AS "<class_name>"  
    [USING JAR|FILE|ARCHIVE '<file_uri>' [, JAR|FILE|ARCHIVE '<file_uri>'] ];  
DROP FUNCTION [IF EXISTS] <function_name>;
```

➤ 列出所有函数、查看函数详情

```
SHOW FUNCTIONS;  
DESCRIBE FUNCTION <function_name>;
```

➤ 开发要点:

1. 自定义UDF函数必须继承 `org.apache.hadoop.hive.ql.exec.UDF`;
2. 自定义UDF函数内需要实现`evaluate()`方法。

`evaluate`的参数个数以及类型都是用户自己定义的;

在使用UDF的时候, Slipstream会调用UDF的`evaluate()`方法。

➤ Demo (自定义UDF函数)

```
package io.transwarp.udf;

import org.apache.hadoop.hive ql.exec.UDF;
import org.apache.hadoop.io.Text;

public class Lower extends UDF {

    public Text evaluate(final Text s) {
        if (s == null) {
            return null;
        }
        return new Text(s.toString().toLowerCase());
    }
}
```

➤ Demo (自定义UDF函数)

```
# 打成jar，并上传到HDFS的/tmp/jars目录中
```

```
# 添加jar包
```

```
add jar hdfs://myhub/tmp/jars/hiveudf.jar;
```

```
list jars;
```

```
# 创建函数
```

```
CREATE TEMPORARY FUNCTION myfunc AS 'io.transwarp.udf.Lower';
```

```
CREATE FUNCTION myfunc2 AS 'io.transwarp.udf.Lower' USING JAR 'hdfs://myhub/tmp/jars/hiveudf.jar';
```

```
# 删除函数
```

```
DROP TEMPORARY FUNCTION myfunc;
```

```
DROP FUNCTION myfunc2;
```

➤ Demo (自定义UDF函数)

```
[root@node3 ~]# beeline -u jdbc:hive2://node2:10000/default
Connecting to jdbc:hive2://node2:10000/default
0: jdbc:hive2://node2:10000/default> add jar hdfs://myhub/tmp/jars/hiveudf.jar;
No rows affected (0.276 seconds)
0: jdbc:hive2://node2:10000/default> list jars;
+-----+
| resource |
+-----+
| /tmp/79bb12f1-11ca-45d2-84b9-0d0998dc2476_resources/hiveudf.jar |
+-----+
1 row selected (0.094 seconds)
0: jdbc:hive2://node2:10000/default> CREATE TEMPORARY FUNCTION myfunc AS 'io.transwarp.udf.Lower';
No rows affected (0.773 seconds)
0: jdbc:hive2://node2:10000/default> select id, name, myfunc(name) as Lower_name from tbl0;
+-----+-----+-----+
| id | name | lower_name |
+-----+-----+-----+
| 1 | zs | zs |
| 1 | z | z |
| 2 | XIaomIng | xiaoming |
+-----+-----+-----+
3 rows selected (1.7 seconds)
0: jdbc:hive2://node2:10000/default> DROP TEMPORARY FUNCTION myfunc;
No rows affected (0.018 seconds)
```

➤ Demo (自定义UDF函数)

```
0: jdbc:hive2://node2:10000/default> CREATE FUNCTION myfunc2 AS 'io.transwarp.udf.Lower' USING JAR
'hdfs://myhub/tmp/jars/hiveudf.jar';
No rows affected (0.19 seconds)
0: jdbc:hive2://node2:10000/default> select id, name, myfunc2(name) as Lower_name from tbl0;
+-----+-----+-----+
| id  | name  | lower_name |
+-----+-----+-----+
| 1   | zs    | zs        |
| 1   | z     | z         |
| 2   | XIaomIng | xiaoming  |
+-----+-----+-----+
3 rows selected (0.159 seconds)
0: jdbc:hive2://node2:10000/default> DROP FUNCTION myfunc2;
No rows affected (0.13 seconds)
```


5
chapter

Hive 性能优化

➤ Hive性能优化

➤ 核心思想：将HiveSQL 当做MapReduce程序去优化

➤ 以下SQL不会转为MapReduce执行：

- select仅查询本表字段
- where仅对本表字段做过滤

➤ 查看执行计划

<https://cwiki.apache.org/confluence/display/Hive/LanguageManual+Explain>

EXPLAIN [EXTENDED | AST | DEPENDENCY | AUTHORIZATION | LOCKS | VECTORIZATION | ANALYZE] query

➤ 查看执行计划

- The Dependency Graph
- The plans of each Stage

```
hive> explain
> from (select explode(split(line, ' ')) as word from docs) w
> insert into table wc
> select word, count(1) as totalword
> group by word
> order by word;
OK
STAGE DEPENDENCIES:
  Stage-1 is a root stage
  Stage-2 depends on stages: Stage-1
  Stage-0 depends on stages: Stage-2
  Stage-3 depends on stages: Stage-0
STAGE PLANS:
  Stage: Stage-1
    Map Reduce
    Map Operator Tree:
      TableScan
        alias: docs
        Statistics: Num rows: 1 Data size: 44 Basic stats: COMPLETE Column stats: NONE
      Select Operator
        expressions: split(line, ' ') (type: array<string>)
        outputColumnNames: _col0
        Statistics: Num rows: 1 Data size: 44 Basic stats: COMPLETE Column stats: NONE
      UDTF Operator
        ... ..
    Reduce Operator Tree:
      Group By Operator
        ... ..
  Stage: Stage-2
    Map Reduce
    Map Operator Tree:
      ... ..
    Reduce Operator Tree:
      Select Operator
        ... ..
  Stage: Stage-0
    Move Operator
    tables:
      replace: false
      table:
        input format: org.apache.hadoop.mapred.TextInputFormat
        output format: org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat
        serde: org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
        name: default.wc
  Stage: Stage-3
    Stats-Aggr Operator
Time taken: 2.621 seconds, Fetched: 86 row(s)
```

➤ Hive运行方式

- 本地模式
- 集群模式

➤ 开启本地模式

- `set hive.exec.mode.local.auto=true;`

➤ 注意:

- `hive.exec.mode.local.auto.inputbytes.max`默认值为128M
 - 表示加载文件的最大值，若大于该配置仍会以集群方式来运行！

➤ 并行计算

➤ 通过设置以下参数开启并行模式:

- `set hive.exec.parallel=true;`

➤ 注意:

- `hive.exec.parallel.thread.number`
 - (一次SQL计算中允许并行执行的job个数的最大值)

➤ 严格模式

➤ 目的：防止用户执行那些可能产生意想不到的不好的效果的查询,从而保护hive的集群

➤ 通过设置以下参数开启严格模式：

- set hive.mapred.mode=strict;
 - （默认为：nonstrict非严格模式）

➤ 查询限制：

1. 对于分区表，必须添加where对于分区字段的条件过滤；
2. order by语句必须包含limit输出限制；
3. 限制执行笛卡尔积的查询。

➤ 排序

1. Order By

- 对于查询结果做全排序，只允许有一个reduce处理
- （当数据量较大时，应慎用。严格模式下，必须结合limit来使用）

2. Sort By

- 对于单个reduce的数据进行排序

3. Distribute By

- 分区排序，经常和Sort By结合使用

4. Cluster By

- 相当于 Sort By + Distribute By
- （Cluster By不能通过asc、desc的方式指定排序规则；
- 可通过 distribute by column sort by column asc|desc 的方式）

➤ Hive Join

➤ Join计算时，将小表（驱动表）放在join的左边

➤ Map Join：在Map端完成Join

➤ 两种实现方式：

1. SQL方式，在SQL语句中添加MapJoin标记（mapjoin hint）

• 语法：

```
SELECT /*+ MAPJOIN(smallTable) */ smallTable.key, bigTable.value  
FROM smallTable JOIN bigTable ON smallTable.key = bigTable.key;
```

2. 开启自动的MapJoin

➤ Hive Join

➤ 开启自动的mapjoin

- set hive.auto.convert.join = true;
 - 该参数为true时, Hive自动对左边的表统计量, 如果是小表就加入内存, 即对小表使用Map join

➤ 相关配置参数:

1. hive.mapjoin.smalltable.filesize;
 - 大表小表判断的阈值, 如果表的大小小于该值则会被加载到内存中运行
2. hive.ignore.mapjoin.hint;
 - 默认值: true; 是否忽略mapjoin hint 即mapjoin标记
3. hive.auto.convert.join.noconditionaltask;
 - 默认值: true; 将普通的join转化为普通的mapjoin时, 是否将多个mapjoin转化为一个mapjoin
4. hive.auto.convert.join.noconditionaltask.size;
 - 将多个mapjoin转化为一个mapjoin时, 其表的最大值

➤ Map端聚合

➤ 通过设置以下参数开启在Map端的聚合：

- set hive.map.aggr=true;

➤ 相关配置参数：

1. hive.groupby.mapaggr.checkinterval:

- map端group by执行聚合时处理的多少行数据（默认：100000）

2. hive.map.aggr.hash.min.reduction:

- 进行聚合的最小比例（预先对100000条数据做聚合，若聚合之后的数据量/100000的值大于该配置0.5，则不会聚合）

3. hive.map.aggr.hash.percentmemory:

- map端聚合使用的内存的最大值

4. hive.map.aggr.hash.force.flush.memory.threshold:

- map端做聚合操作是hash表的最大可用内容，大于该值则会触发flush

5. hive.groupby.skewindata

- 是否对GroupBy产生的数据倾斜做优化，默认为false

➤ Map & Reduce 数量

➤ Map数量相关的参数

1. `mapred.max.split.size`
 - 一个split的最大值，即每个map处理文件的最大值
2. `mapred.min.split.size.per.node`
 - 一个节点上split的最小值
3. `mapred.min.split.size.per.rack`
 - 一个机架上split的最小值

➤ Reduce数量相关的参数

1. `mapred.reduce.tasks`
 - 强制指定reduce任务的数量
2. `hive.exec.reducers.bytes.per.reducer`
 - 每个reduce任务处理的数据量
3. `hive.exec.reducers.max`
 - 每个任务最大的reduce数

➤ JVM重用

➤ 适用场景：

1. 小文件个数过多
2. task个数过多

➤ 通过 set mapred.job.reuse.jvm.num.tasks=n; 来设置 (n为task插槽个数)

➤ 缺点：

- 设置开启之后，task插槽会一直占用资源，不论是否有task运行，直到所有的task即整个job全部执行完成时，才会释放所有的task插槽资源！



Q&A

TRANSWARP
星环科技